HOSTEL HIVE

Frontend Architecture & Technology Stack

A student-focused platform for discovering hostels, messes, cafeterias, colleges, and student community content.

Project Showcase Document

Prepared by: Mohd Anas
report.data@ashokauto.com

October 2026

1. Project Overview

Hostel Hive is a student-focused web platform for discovering hostels, messes, cafeterias, colleges, and student community content. This document describes the **public-facing frontend** of the application — its technology stack, architecture, and feature set — built with a modern, server-rendered React framework.

The frontend communicates with a live, read-only backend API for listings and college data, and is designed with a component-driven architecture for reusability, performance, and a responsive user experience across devices.

2. Technology Stack

Layer	Technology	Purpose
Framework	Next.js 16 (App Router)	Full-stack React framework: routing, SSR, server actions
UI Library	React 19	Component-based, declarative UI rendering
Styling	Tailwind CSS 4	Utility-first CSS for rapid, consistent styling
CSS Tooling	@tailwindcss/postcss, PostCSS	Tailwind integration into the build pipeline
Data Layer	Native Fetch API	REST API integration with the Hostel Hive backend
Linting	Oxlint	Fast Rust-based JavaScript/React linter for code quality
Package Manager	npm	Dependency management
Language	JavaScript (JSX)	Component and application logic

Why Next.js (App Router)?

- File-system based routing with nested layouts and route-level loading/error states
- Server Components by default, reducing client-side JavaScript and improving performance
- Server Actions for handling mutations (e.g. authentication) without separate API routes
- Built-in support for SEO metadata, image optimization, and fast page transitions

Why Tailwind CSS?

- Utility classes enable rapid UI development directly in JSX without context-switching to CSS files
- Built-in responsive design utilities for consistent mobile and desktop layouts
- Small production CSS bundles via automatic unused-style purging

3. Application Architecture

The codebase follows a clear separation of concerns, organised into four primary directories:

Directory	Responsibility
app/	Route pages, nested layouts, and per-route loading/error UI (App Router)
components/	Reusable UI and feature components, grouped by domain (home, auth, explore, etc.)

lib/	API integration logic and data-fetching utilities per domain (listings, messes, auth, community)
data/	Placeholder / demo data used for prototype features pending full backend integration

Component Organisation

- **layout/** — Navbar and Footer shared across all pages
- **home/** — Hero, Category Grid, Featured Listings, Testimonials, FAQ, Trust and CTA sections
- **explore/** and **listing/** — Search filters, listing cards, booking actions, location maps
- **messes/** and **colleges/** — Domain-specific cards and filters
- **community/** — Feed, post cards, and post creation modal
- **auth/** — Login and registration forms
- **ui/** — Shared, generic presentational components (e.g. ListingCard)

4. Key Features

- **Explore listings** — Browse hostels, messes, and cafeterias in one unified interface
- **Search & filters** — Client-side search, filtering, and sorting across fetched listings
- **Listing details** — Type-specific listing pages with pricing and location information
- **College directory** — Browse colleges with city-based filtering
- **Student community** — A social feed with posts and a post-creation interface
- **Authentication** — Login and registration flows using Next.js Server Actions
- **Responsive design** — Fully adapted layouts for desktop and mobile screen sizes
- **Robust error handling** — Dedicated loading, error, and not-found states per route

5. Application Routes

Route	Purpose
/	Homepage
/explore	Explore hostels, messes, and cafeterias
/messes	Mess discovery page
/colleges	College directory
/community	Student community interface
/listings/[id]	Dynamic listing detail pages
/login, /register	Authentication pages
/about, /contact	Informational pages

6. Backend Integration

The frontend consumes a live, read-only REST API, normalising responses into a consistent shape for shared listing components.

Base URL:

```
https://hostel-hive-backend.onrender.com
```

Integrated Endpoints:

- GET /hostels
- GET /messes
- GET /cafeterias
- GET /colleges
- Individual detail endpoints for hostel, mess, and cafeteria listings

Explore fetches up to 100 records per category in a single request; search, filtering, and sorting are then performed client-side against the fetched dataset. Optional fields such as ratings, review counts, and images are handled defensively when absent from the API response.

7. Development Workflow

Command	Description
npm run dev	Start the development server (localhost:3000)
npm run build	Create an optimised production build
npm run start	Serve the production build
npm run lint	Run Oxlint for static code analysis

Current Development Status

- Listings and college directory are integrated with the live backend API
- Community content currently uses sample/demo data for prototyping
- Authentication flows are implemented; bookings and payments are not yet completed features
- Production SEO configuration and CORS verification remain under review